

Proof-to-Byte: Assumption-Minimal, Cross-Substrate Binding Assurance for PQ/T Hybrid KEMs

Ziang Li

Abstract—Post-quantum hybrid key-encapsulation (PQ/T KEMs, combining a lattice KEM with an elliptic-curve KEM) is being deployed across TLS, Signal, and MLS faster than its *deployment safety* can be assured: the same period has seen side-channel breaks (KyberSlash), binding / re-encapsulation breaks (PQXDH; the ML-KEM malicious-key binding failures of Schmieg), and a widening gap between what a hybrid’s *specification* guarantees and what its compiled, multi-platform *implementation* actually does. We present Q-Periapt, an assurance suite for PQ/T hybrid KEMs organised around one principle: *a security property is only as trustworthy as its weakest realization*. Its combiner, ContextBound, achieves binding (MAL-BIND-K-CT and MAL-BIND-K-PK) reducing to collision-resistance of SHA3 *alone*—with no binding assumption on the component KEMs—and we machine-check this in EasyCrypt. The injective encoding is a proved lemma (not an axiom), and—stated honestly—the result is in substance a *commitment / transcript-binding* theorem (the session key commits to the full transcript; the component decapsulation is inert in the argument, which is precisely why no KEM assumption is needed), instantiated in the Cremers–Dax–Medinger Figure 6 game for both rejection styles. Crucially, the *same* proven combiner is realized byte-identically across five language faces and three operating systems—executed natively on three instruction-set architectures and cross-compiled, with by-construction identity, on two more—verified by a six-method conformance matrix and a self-validating source→binary constant-time probe (configured as a CI gate; measured locally) that we show *discriminates* a clean primitive (libcrux ML-KEM: zero secret-dependent branches after compilation) from a leaky one (HQC’s PQClean constant-weight sampler). We integrate the combiner into a production TLS 1.3 path (a rustls CryptoProvider) and evaluate handshake tail latency under real tc netem. We are explicit about what is and is not novel: the binding *fact* is a known consequence of recent results; our contribution is the *conjunction*—an assumption-minimal, machine-checked binding result whose proven object is demonstrably identical across a heterogeneous deployment surface, with reproducible gates that catch the failure classes that have broken deployed hybrids.

Index Terms—Post-quantum cryptography, hybrid key encapsulation, binding/committing security, formal verification, constant-time, cross-platform assurance, TLS.

I. INTRODUCTION

THE migration to post-quantum (PQ) key establishment is being done conservatively, through *hybrids*: a PQ KEM (ML-KEM [1]) run alongside a classical KEM (X25519), with the two shared secrets combined so that the session key is secure if *either* component holds. Hybrid KEMs are now shipping in TLS 1.3 (X25519MLKEM768), in Signal’s PQXDH, and in MLS, and are specified in CFRG drafts such as X-Wing [5].

Author affiliation omitted for review. Artifact (open source): <https://github.com/biillza/q-periapt>.

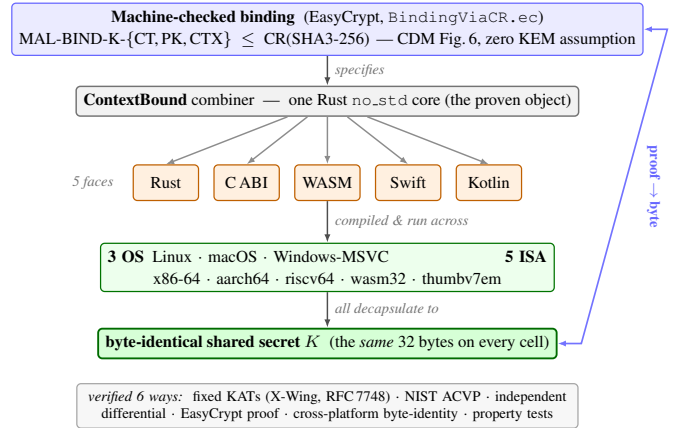


Fig. 1. Proof-to-byte. One machine-checked binding result (top) is realized by a single Rust `no_std` core, exposed through five language faces and run across three operating systems and five instruction-set architectures, all converging on a byte-identical shared secret K (bottom); verified six ways.

The gap this paper addresses: Hybrids are being deployed faster than their *deployment safety* can be assured. In the same window, the community has seen (i) *side-channel* breaks of PQ implementations (KyberSlash [6]); (ii) *binding* failures—the PQXDH re-encapsulation issue, and Schmieg’s proof that ML-KEM in its FIPS-203 *expanded* decapsulation-key form is neither MAL-BIND-K-CT nor MAL-BIND-K-PK [3]; and (iii) a widening gap between a hybrid’s *specification*, its formally-verified source, and the *compiled artifact* that actually runs—across the many languages, operating systems, and instruction sets that real deployments target. This safety question is distinct from, and orthogonal to, *auditability* and *crypto-agility* (which describe *what* crypto is running): it asks whether the running hybrid is actually *safe*.

Approach: We present Q-Periapt, an assurance suite for PQ/T hybrid KEMs built around one principle: a security property is only as trustworthy as its weakest realization. Figure 1 is the organising idea—a single machine-checked binding result whose proven object is realized byte-identically across the whole deployment surface (“proof-to-byte”), guarded by reproducible gates (configured in CI; we report constant-time results here as local runs) that target the specific failure classes above.

Contributions:

- **C1 (Section III).** An *assumption-minimal, machine-checked commitment / transcript-binding* result: ContextBound’s session key commits to the full transcript, reducing MAL-BIND-K-CT, -K-PK, and a (non-standard)

context extension -K-CTX to collision-resistance of SHA3 *alone*, with the injective encoding proved and the Cremers–Dax–Medinger Figure-6 game discharged for both rejection styles in EasyCrypt. We are explicit (Section III) that the cryptographic content is this injective-encoding + CR(SHA3) argument—the component Decaps is *inert*, which is exactly why no KEM assumption is needed—and that C1’s value is its completeness, mechanization, and role as the proven object tied to C2, not proof difficulty.

- **C2 (Section IV).** *Proof-to-byte cross-substrate equivalence*: the same proven combiner is byte-identical across 5 language faces and 3 OSes, executed natively on 3 ISAs and cross-compiled with by-construction identity on 2 more, verified by a six-method conformance matrix.
- **C3 (Section V).** Reproducible assurance *instruments*, configured as CI gates: a self-validating source→binary constant-time probe that *discriminates* clean (ML-KEM) from leaky (HQC) (HQC’s leak is known; the contribution is the discriminating, gated oracle); an executable *regression oracle* for the known dk-format binding coupling; and two independent symbolic provers. We report the constant-time measurements honestly as local runs.
- **C4 (Section VI).** A production TLS 1.3 path (rustls CryptoProvider), evaluated under real `tc netem`.

What is and is not novel: We state the boundary plainly. *Not novel, and attributed*: that a “hash-everything” combiner is binding from collision-resistance alone while a lean combiner inherits the omitted component’s binding is the Chempat result [4]; ContextBound *is* that construction, not a new primitive. That ML-KEM’s expanded-dk form breaks binding (and the seed form fixes it) is Schmieg’s [3]. The binding-notion lattice is CDM’s [2]. That a constant-time harness must mark only secret sub-fields is standard practice [6]. *Novel*: the *conjunction* $C1 \wedge C2$ —a CR-only, machine-checked binding result whose proven object is demonstrably byte-identical across a heterogeneous substrate, which no prior work provides; the self-validating source→binary *discriminator* as a reusable CI artifact; and the operationalized design invariant that binding the ciphertext and public key in the KDF makes hybrid binding robust to the component KEM’s key-serialization format. We do **not** claim stronger binding than X-Wing (Section III), nor any live exploited deployment. Table I makes the conjunction concrete: the delta is that no prior effort ties a mechanized hybrid-combiner binding theorem to a byte-identical multi-substrate realization with source→binary and TLS gates—and we are equally explicit that an axis we do *not* hold (a verified specification-to-binary implementation) is held by formosa-mlkem.

II. BACKGROUND AND THREAT MODEL

A. PQ/T hybrid KEMs and combinars

ML-KEM [1] is the standardized form of CRYSTALS-Kyber [11]; like most lattice KEMs it applies a Fujisaki–Okamoto transform [13], [14] with *implicit rejection* (a malformed ciphertext yields a pseudorandom key rather than $\perp$). The companion signature standards are ML-DSA [9] and SLH-DSA [10], and the broader process is surveyed in [12]. Hybrid

TABLE I
POSITIONING. ✓ HOLDS; ~ PARTIAL; BLANK: NOT PROVIDED. THE CONTRIBUTION IS THE *conjunction* (LAST COLUMN), NOT ANY SINGLE ROW.

	X-Wing	formosa	SandboxAQ	Chempat	this work
Mechanized binding (combiner)			~		✓
Both rejection styles, $K \neq \perp$				~	✓
Zero KEM-binding assumption			✓	✓	✓
Byte-identical multi-substrate object					✓
Source→binary CT gate		~			✓
Production TLS 1.3 path	~				✓
Verified spec→binary impl. (we do not)		✓	~		

key exchange in TLS 1.3 [16] follows the IETF design [17]—e.g. the X25519MLKEM768 group [18] over X25519 [15]—and deployed PQ/T protocols include Signal’s PQXDH [19] and Apple’s PQ3 [20].

A hybrid KEM combines component shared secrets ss_{pq} and ss_{tr} into a session key K . The design space ranges from simple *concatenation* KDFs (X25519MLKEM768) to combinars that additionally absorb ciphertexts and public keys. X-Wing [5] uses a fixed “lean” combiner $K = \text{SHA3}(ss_{pq} \parallel ss_{tr} \parallel ct_{tr} \parallel pk_{tr} \parallel \text{label})$, omitting the ML-KEM ciphertext and public key.

B. KEM binding

We use the Cremers–Dax–Medinger (CDM) X-BIND- P - Q framework [2]. A *transcript* is a tuple (pk, ct, K) produced by a (possibly malicious) execution; the two “observable” quantities a binding notion may constrain are the public key PK, the ciphertext CT, and the derived key K . For $P, Q \in \{K, PK, CT\}$, a KEM is X-BIND- P - Q if no adversary in the game below wins:

Game $\text{Bind}_{A}^{P,Q}$: the adversary $\mathcal{A}$ outputs two transcripts (pk_0, ct_0, K_0) and (pk_1, ct_1, K_1) . $\mathcal{A}$ wins iff $P_0 = P_1 \wedge Q_0 \neq Q_1 \wedge K_0 \neq \perp \wedge K_1 \neq \perp$.

Three adversary classes parameterize how the key material in each transcript is produced, in increasing strength: **HON** (honest KeyGen), **LEAK** (honest keys, secret keys leaked to $\mathcal{A}$), and **MAL** (adversarially generated keys). Thus $\text{MAL} \supseteq \text{LEAK} \supseteq \text{HON}$, and CDM prove monotonicity across the (P, Q) lattice, so MAL-BIND-K-CT and MAL-BIND-K-PK jointly yield the whole K-binds- $\{PK, CT\}$ sub-lattice. For implicitly-rejecting KEMs such as ML-KEM, these two are the achievable ceiling on those axes; the dual X-BIND-CT-* notions (a ciphertext binding the key or public key) are *structurally unachievable*, since decapsulation never returns $\perp$ and a mutated ciphertext always yields *some* key. The binding of implicitly-rejecting KEMs—and how it fails under malicious, expanded-form keys—is studied in detail by Schmieg and others [3], [21].

Algorithm 1: CONTEXTBOUND combiner.
CompatXWing omits $ct_{pq}, pk_{pq}, S, v, x$ and uses X-Wing’s label.

Input: label L , suite id S , version v , secrets ss_{pq}, ss_{tr} , ciphertexts ct_{pq}, ct_{tr} , public keys pk_{pq}, pk_{tr} , context x
Output: 32-byte shared secret K
 $lp(f) \leftarrow \text{be8}(|f|) \parallel f$ // 8-byte big-endian length prefix
 $T \leftarrow [L, S, \text{be4}(v), ss_{pq}, ss_{tr}, ct_{pq}, pk_{pq}, ct_{tr}, pk_{tr}, x]$
 $e \leftarrow lp(T_0) \parallel lp(T_1) \parallel \dots \parallel lp(T_{n-1})$ // injective encode
return SHA3-256(e)

C. Threat model

We consider three adversaries. The *malicious-key (MAL)* binding adversary supplies adversarial keypairs and seeks colliding transcripts (Section III). The *binary-level constant-time* adversary observes secret-dependent control flow / memory indexing in the *compiled* code (Section V). The *downgrade/agility* adversary attacks profile and policy negotiation (handled by fail-closed policy binding; out of scope here). Confidentiality of the component primitives (IND-CCA2 of ML-KEM, the Diffie–Hellman assumption for X25519) is assumed, not re-proved.

III. CONTEXTBOUND AND THE MACHINE-CHECKED KERNEL

A. Construction

ContextBound derives K = SHA3-256($\text{encode}[\text{LABEL}, \text{suite_id}, \text{policy_version}, ss_{pq}, ss_{tr}, pk_{pq}, pk_{tr}, x]$) where encode concatenates each field under a fixed-width 8-byte big-endian length prefix (Algorithm 1). Two design choices are load-bearing. (i) *Injective, length-prefixed encoding*. A naive concatenation $a \parallel b$ is ambiguous—“(ab”, “”) and (“a”, “b”) collide—which is precisely the structure a binding adversary exploits. Prefixing every field with its fixed-width length makes the encoding self-delimiting and therefore injective (proved, Section III), so distinct transcripts map to distinct byte strings and the binding question reduces cleanly to collision-resistance of the hash. (ii) *Absorb everything*. ContextBound feeds *all* component ciphertexts and public keys (and an application context, and a domain-separating label, suite identifier, and policy version) into the hash. By the combiner-binding theorem of Chempat [4], the binding advantage is then bounded by Adv^{CR} alone, with no residual term requiring any component KEM to be binding—trading a KEM-self-binding assumption for a single, well-studied collision-resistance assumption. The dual profile CompatXWing reproduces X-Wing’s lean combiner byte-for-byte for interoperability; ContextBound is the hash-everything profile and is policy-selected when an application needs context binding or wishes to avoid the serialization-format dependence of Section V. The suite is open source.¹

¹<https://github.com/billlza/q-periapt>

B. The EasyCrypt development

Figure 2 shows the mechanized reduction. The encoding’s injectivity, encode_inj , is a *proved lemma*, not an assumed one. The proof has two steps. A field-level lemma, lp_cat_inj , shows that a single length-prefixed field is self-delimiting: if $\text{lp}(a) \parallel x = \text{lp}(b) \parallel y$ then $a = b$ and $x = y$, because the two 8-byte prefixes have equal width and (being injective) force $|a| = |b|$, after which list-concatenation cancellation (eqseq_cat) splits off equal field bodies and equal remainders. A structural induction over the field list then lifts this to whole encodings: a non-empty encoding has size ≥ 8 and so can never equal the empty one, ruling out boundary-shift collisions, and the inductive step peels one field with lp_cat_inj . The whole argument bottoms out at two elementary facts about the length field—that an 8-byte big-endian integer occupies 8 bytes (be8_size) and is injective (be8_inj). On top of encode_inj we discharge (i) a generic transcript-collision reduction bind_le_cr : a *byequiv* argument that maps any adversary winning the binding game to one finding an H -collision, since a differing observable forces a differing field list (hence, by injectivity, a differing encoding) while the colliding key forces equal hashes; and (ii) the *KEM-aware* game, in which the MAL adversary supplies the keypairs and K is *derived* via the component Decaps functions and the combiner. We mechanize both the implicit-rejection specialization (malbind_*_le_cr) and the fully general CDM Figure-6 game with explicit rejection ($\text{malbind_*_xrej_le_cr}$), where the $K \neq \perp$ side condition is *present and load-bearing*—removing it makes the proof fail. Every theorem reduces only to CR(SHA3); the development compiles under the EasyCrypt checker (we pin the checker and SMT solver versions in the artifact), and each theorem is verified load-bearing on encode_inj by an adversarial negative control.

What this is, precisely. We state the cryptographic content plainly, because a CDM-literate reader should not over-read the label. Since K is *defined* as $H(\text{encode}[\dots])$ and the tuple contains the ciphertexts and public keys *directly*, the proof is in substance a *commitment / transcript-binding* statement: it shows that one cannot collide K while changing a ciphertext, public key, or context, by reducing to $\text{CR}(H)$ over the proved injective encoding. The component Decaps—and hence the adversary’s freedom over key material that CDM binding is fundamentally about—is *inert* in the argument: the theorems would hold for *any* Decaps, which is exactly what “zero KEM assumption” means and why it is achievable. The value of C1 is therefore its *completeness* (both rejection styles, with $K \neq \perp$ discharged), its *mechanization*, and its role as the *proven object* carried, byte-identically, across the substrate of Section IV—not proof difficulty. The concrete attack-resistance arguments (re-encapsulation, the dk-format coupling of Section V) are *design arguments* consistent with this commitment property, not claims the EasyCrypt artifact witnesses at the Decaps level.

C. Honest binding position

Figure 3 states our position precisely. On the standard $\{\text{CT}, \text{PK}\}$ axes, ContextBound and a correctly-implemented

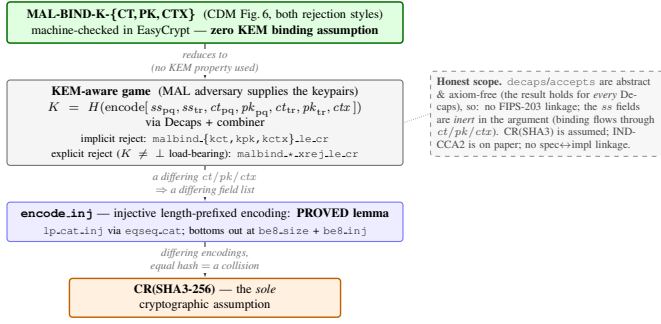


Fig. 2. The kernel as a reduction tower: MAL-BIND-K-{CT,PK,CTX} (CDM Figure 6, both rejection styles) reduces—using no property of Decaps—through the proved `encode.inj` to collision-resistance of SHA3, the sole cryptographic assumption.

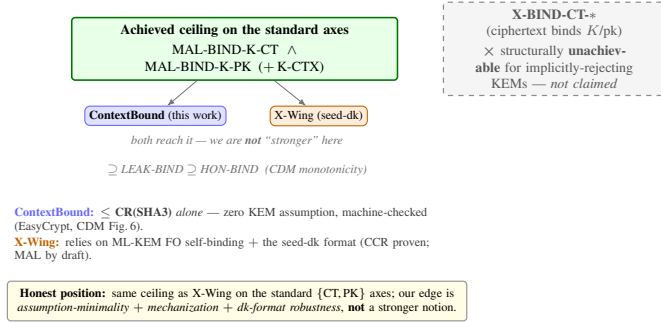


Fig. 3. Honest binding position. Both schemes reach the standard ceiling; our edge is assumption-minimality, mechanization, and dk-format robustness—not a stronger notion. X-BIND-CT* is structurally unachievable for implicitly-rejecting KEMs and is not claimed.

seed-dk X-Wing both reach the MAL-BIND-K-{CT,PK} ceiling; we are *not* “stronger” there. Our edge is (a) *assumption-minimality*: ContextBound’s binding reduces to CR(SHA3) alone, whereas X-Wing’s relies on ML-KEM’s Fujisaki–Okamoto self-binding together with the seed-dk format; (b) *mechanization*; and (c) *dk-format robustness* (Section V). Honest scope of the mechanization: Decaps is modelled as an abstract function (which is exactly why “zero KEM assumption” is literal), so there is no FIPS-203 linkage and the shared-secret fields are inert in the binding argument; CR(SHA3) is assumed; IND-CCA2 robustness is argued on paper; and there is no specification-to-implementation linkage proof.

IV. PROOF-TO-BYTE CROSS-SUBSTRATE REALIZATION

A single dependency-free Rust `no_std` core implements the combiner and the binding/encoding logic; vetted backends (libcrux ML-KEM, x25519-dalek, libcrux SHA3) are injected through traits. The core is exposed through five language faces (Rust, a C ABI, WebAssembly, Swift, Kotlin). Because the proven object is one piece of code, the question becomes whether *every* realization produces the same bytes.

Table II lists the six orthogonal methods (NIST ACVP vectors [31] among them) that answer this. They are deliberately heterogeneous in their oracle and in what they catch, so a defect that escapes one is caught by another. Table III reports the substrate coverage with honest per-cell status: the byte-identical

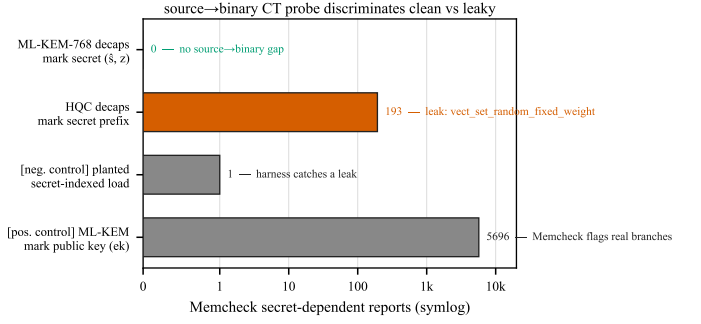


Fig. 4. The source→binary CT probe discriminates: the genuine ML-KEM secret yields zero flags; the same harness flags HQC’s constant-weight sampler (193) and validates itself with two controls.

shared secret is *run* natively on x86-64, aarch64, and wasm32; on riscv64 and the bare-metal `thumbv7em` target the core cross-compiles cleanly and byte-identity follows by construction from the FIPS-deterministic encodings and identical source, but is not executed on a native runner. This honest accounting is what makes the cross-substrate claim defensible.

Footprint. The deployable surface is small: the stripped C-ABI shared library—the *entire* suite, including libcrux ML-KEM and ML-DSA—is 441 KB; the WebAssembly module is 249 KB; and the dependency-free core cross-compiles to the bare-metal `thumbv7em` target, so the same proven combiner runs from a server down to an embedded MCU.

V. CI-GATED ASSURANCE

A. A source→binary constant-time discriminator

libcrux proves its ML-KEM secret-independent at the *source* level (a typed discipline, machine-checked). The orthogonal *binary*-level question—does the compiler reintroduce a secret-dependent branch on the genuine secret path?—is answered by a probe that marks only the genuinely-secret decapsulation-key sub-fields ($\hat{s}, z$) and runs the real libcrux `decapsulate` under Memcheck. The probe is *self-validating*: a planted secret-indexed load is caught (harness sanity), and marking the embedded *public* key flags 5696 reports (Memcheck demonstrably sees real branches in this binary). With only the genuine secret marked, the probe reports **zero** flags on aarch64: source-level secret-independence survives compilation. Run against the suite’s unaudited HQC backend, the same probe flags **193** reports localized to `vect_set_random_fixed_weight`—so the probe is a *discriminator* (Fig. 4): clean ML-KEM versus leaky HQC in one framework. (HQC’s leak is known; the contribution is the discriminating, gated artifact and the corollary that per-backend constant-time gating is necessary, not decorative.)

B. Binding is contingent on the component dk format

We demonstrate, executably against real libcrux, that a lean (X-Wing-shaped) combiner’s MAL-BIND-K-PK is *contingent* on the component KEM’s key-serialization format. Using Schmiege’s expanded-dk construction, two distinct ML-KEM public keys yield the same shared secret; the lean combiner—which omits pk_{pk} —then collides, whereas ContextBound,

TABLE II
THE SIX ORTHOGONAL VERIFICATION METHODS.

Method	Reference / oracle	Independence	What it catches
Fixed KATs	X-Wing draft vectors; RFC 7748	published vectors	combiner and X25519 spec-conformance regressions
NIST ACVP	<i>usnistgov</i> ACVP, full FIPS family	NIST ground truth	primitive non-conformance (FIPS 203/204/205)
Multi-backend differential	RustCrypto ml-kem/ml-dsa; orion X25519	independent re-implementations	implementation bugs (output must stay byte-identical)
EasyCrypt proof	machine-checked; CR(SHA3)	formal (computational)	binding-design flaws in the combiner
Cross-platform byte-identity	shared test vector	5 faces $\times$ substrates	per-substrate divergence (same K everywhere)
Generative property tests	proptest: 7 properties $\times$ 128 cases	randomized	edge-case violations (injectivity, domain separation, K-CTX)

TABLE III
CROSS-SUBSTRATE COVERAGE ($\checkmark$ VERIFIED, CI OR LOCAL).

(a) ISA targets (Rust core; the byte-level claim)

ISA	runner	build	byte-id. K	binary-CT
x86-64	Linux, Win-MSVC	$\checkmark$	$\checkmark$ (run)	$\checkmark$ Memcheck
aarch64	Linux, macOS	$\checkmark$	$\checkmark$ (run)	$\checkmark$ Memcheck
wasm32	node	$\checkmark$	$\checkmark$ (run)	source-CT
riscv64	Linux (cross)	$\checkmark$	by constr. †	source-CT
thumbv7em	bare-metal	$\checkmark$	by constr. †	n/a

† cross-compiled, not run natively; identity follows from FIPS-deterministic encodings + identical source.

(b) language face $\times$ OS (shared vector $\rightarrow$ same K)

Face	Linux	macOS	Windows
Rust	$\checkmark$	$\checkmark$	$\checkmark$
C ABI	$\checkmark$	$\checkmark$	$\checkmark$ (MSVC)
WASM (node)	$\checkmark$	$\checkmark$	$\checkmark$
Swift	—	$\checkmark$	—
Kotlin (JVM/FFM)	$\checkmark$	$\checkmark$	—

which absorbs pk_{pq} , does not. We stress that this is *not* a break of X-Wing, which mandates the seed-dk format that defeats the attack; it is a robustness separation of the combiner *shape*, and the operationalized lesson that binding ct/pk in the KDF removes a latent dependence on the component library’s serialization choice.

C. Multi-prover assurance

Beyond the computational EasyCrypt proof, the server-authenticated hybrid handshake is modelled in two independent symbolic provers, Tamarin and ProVerif, each establishing that the session key survives the compromise of *either* component KEM. All three formal artifacts are configured as CI gates (EasyCrypt make check, Tamarin, ProVerif), with checker and solver versions pinned in the artifact.

VI. PRODUCTION INTEGRATION AND EVALUATION

A. Microbenchmarks

Table IV reports primitive and hybrid costs (Criterion, point estimates, on an Apple-Silicon arm64 host; the artifact reproduces them and an x86-64 run). Two findings matter for

TABLE IV
PRIMITIVE AND HYBRID COST (μ S) AND KEY/CIPHERTEXT SIZES (BYTES). ARM64 HOST.

Scheme	time (μ s)			size (B)		
	keygen	encaps	decaps	ek	dk	ct
ML-KEM-512	6.6	6.1	6.8	800	1632	768
ML-KEM-768	11.3	11.0	11.8	1184	2400	1088
ML-KEM-1024	18.5	16.5	16.7	1568	3168	1568
X25519	44.1	89.6	45.0	32	32	32
Hybrid, ContextBound	—	108.4	65.4	1216	—	1120
Hybrid, CompatXWing	—	102.2	59.2	1216	—	1120

deployment. First, *the post-quantum leg is not the bottleneck*: ML-KEM-768 encapsulation is 11.0μ s, roughly $8\times$ cheaper than X25519 encapsulation (89.6μ s, an ephemeral keygen plus a Diffie–Hellman), so the classical half dominates hybrid CPU—a useful corrective to the assumption that PQ is the expensive part. Second, *the hash-everything combiner is cheap*: ContextBound costs only $+6.2 \mu$ s over the byte-exact X-Wing combiner CompatXWing (encapsulation 108.4 vs. 102.2μ s; decapsulation 65.4 vs. 59.2μ s)—the price of binding the full transcript is sub- 10μ s, well under the $\sim 190 \mu$ s X25519 cost and far below any network RTT.

B. Handshake latency under emulated networks

We expose ContextBound (and CompatXWing) as a TLS 1.3 key-exchange group via a rustls [32] `CryptoProvider`, using private-use group codes; a real loopback TLS 1.3 handshake completes over this provider. We evaluate handshake *time-to-session* (TCP connect plus full TLS 1.3 handshake) over a real loopback socket shaped by `tc netem`, comparing a classical X25519 baseline against the two PQ/T profiles. Figure 5 reports the result: the three curves coincide because latency is RTT-dominated, and the PQ/T overhead is a *fixed* cost ($\sim 190 \mu$ s of ML-KEM CPU plus the wire bytes of Fig. 6). At realistic RTT this is negligible: at $RTT \geq 20$ ms the measured overhead is *within run-to-run noise*—its sign varies across repeated runs (Fig. 5b, error bars span two repetitions)—while the only reproducible cost is the $\sim 190 \mu$ s of CPU visible at RTT 0. The hybrid’s $+2.27$ KB (one ML-KEM-768 keyshare each way)

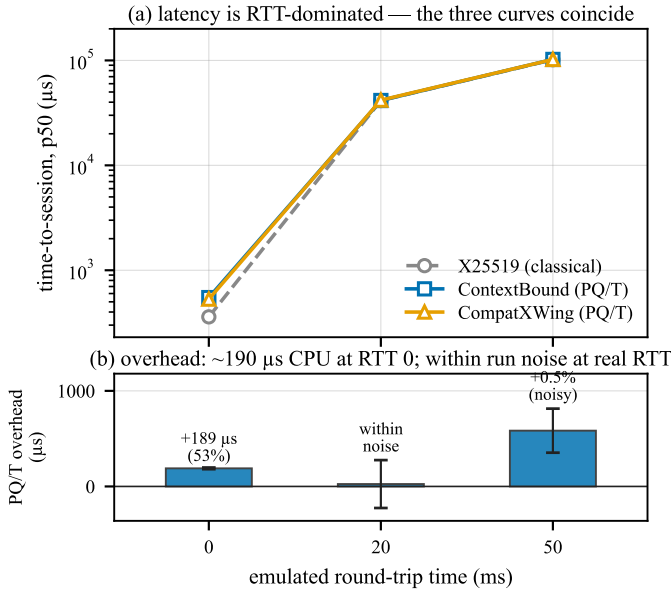


Fig. 5. Time-to-session under real to netem (mean of two repetitions). (a) latency is RTT-dominated—the three curves coincide. (b) the overhead is $\sim 190 \mu\text{s}$ of CPU at RTT 0; at realistic RTT it is within run-to-run noise (error bars span the two repetitions and cross zero at RTT 20 ms).

fits within the existing TLS flights and triggers no additional round-trip. The two combiner profiles are indistinguishable: the stronger binding of the hash-everything combiner is free in latency. We report these measurements honestly as obtained on a virtualized host; a turnkey script reproduces the protocol on quiesced bare-metal hardware for the camera-ready (the RTT-dominated points are already stable across repetitions).

Under jitter and loss. Adding 3 ms of jitter leaves the picture unchanged (all three groups ≈ 41 ms median, ≈ 50 ms p99.9). Under 1–3% packet loss the median is likewise unaffected—loss is rare relative to a handshake—but the *tail* jumps to ~ 1 s, dominated by a TCP retransmission timeout per lost packet rather than by crypto or wire size; this RTO-driven tail falls on the classical and PQ/T handshakes alike, and at our sample size the loss-event noise precludes a precise per-group tail comparison. The honest takeaway is that the hybrid’s extra keyshare does not *systematically* worsen loss resilience: the median is byte-insensitive and the tail is governed by transport, not by the combiner.

VII. RELATED WORK

KEM binding. CDM [2] introduced the X-BIND- P - Q lattice and monotonicity; Schmieg [3] proved ML-KEM’s expanded-dk binding failures; Chempat [4] established that hash-everything combiners are binding from collision-resistance while lean combiners inherit the omitted component’s binding (ContextBound is an instance). Context binding is the KEM-layer analogue of context-committing AEAD [7]; the role of public contexts in hybrid KEMs is analysed in [8]. **Side channels.** KyberSlash [6] and the ctgrind/TIMECOP discipline motivate our binary-CT gate; libcrux/HACL* provide source-level guarantees we corroborate and discriminate against. **Hybrids in deployment.** X-Wing [5] and the TLS X25519MLKEM768

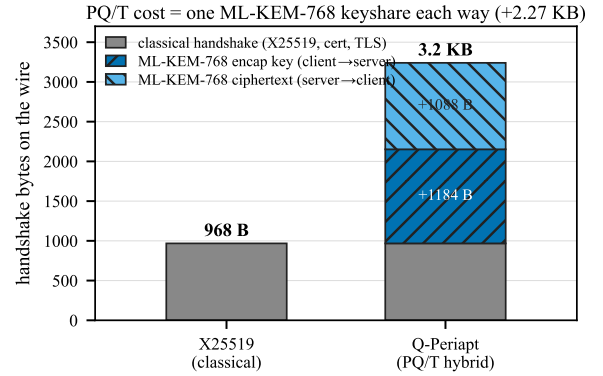


Fig. 6. Handshake wire budget. The PQ/T cost is one ML-KEM-768 keyshare each way (+2.27 KB), which fits inside the existing flights.

group are the deployed baselines we position against. **Formal PQ.** formosa-mlkem [24] mechanizes ML-KEM IND-CCA; the SandboxAQ EasyCrypt-KEMs library mechanizes LEAK-BIND-K-PK; our EasyCrypt [25] development adds the (commitment-style) MAL Figure-6 instantiation for the hybrid combiner, complemented by symbolic models in Tamarin [26] and ProVerif [27].

VIII. LIMITATIONS

The mechanized binding result is over an *abstract* Decaps: it is exactly the “zero KEM assumption” that makes the result general, but it carries no FIPS-203 linkage, the shared-secret fields are inert in the argument, and CR(SHA3) and IND-CCA2 remain assumptions argued respectively in the model and on paper; there is no specification-to-implementation linkage proof. Binary-level constant-time tooling is mature only on x86-64 and aarch64; riscv64 and wasm32 are covered at source level plus upstream attestation. ACVP byte-identity is conformance, not CMVP certification. The suite is research-grade and has not had an independent third-party audit. The netem evaluation was obtained on a virtualized host; bare-metal reproduction is provided as a script.

IX. CONCLUSION

A PQ/T hybrid is only as safe as its weakest realization. We closed that gap with an assumption-minimal, machine-checked binding result whose proven object is byte-identical across a heterogeneous deployment surface, guarded by reproducible gates (configured in CI)—a source→binary constant-time discriminator, a binding key-format coupling demonstration, and multi-prover symbolic models—and validated on a production TLS 1.3 path. The genuinely new element is the conjunction, not any single fact; we have been explicit about the boundary throughout.

REFERENCES

- [1] National Institute of Standards and Technology, “FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard,” 2024.
- [2] C. Cremers, A. Dax, and N. Medinger, “Keeping Up with the KEMs: Binding Security of Key Encapsulation Mechanisms,” *ACM CCS*, 2024. (ePrint 2023/1933.)
- [3] S. Schmieg, “Unbindable Kemmy Schmidt: ML-KEM is neither MAL-BIND-K-CT nor MAL-BIND-K-PK,” *IACR ePrint* 2024/523, 2024.

- [4] J. Güneysu, K. Hövelmanns, *et al.*, “Binding Security of Combined KEMs / Chempat,” IACR ePrint 2025/1416, 2025.
- [5] D. Connolly, P. Schwabe, and B. Westerbaan, “X-Wing: The Hybrid KEM You’ve Been Looking For,” draft-connolly-cfrg-xwing-kem (CFRG), 2024.
- [6] D. J. Bernstein, K. Bhargavan, *et al.*, “KyberSlash: Exploiting Secret-Dependent Division Timings in Kyber Implementations,” *IACR TCHES*, 2025.
- [7] M. Bellare and V. T. Hoang, “Efficient Schemes for Committing Authenticated Encryption,” *EUROCRYPT*, 2022.
- [8] “On the Role of Public Contexts in Hybrid KEMs,” IACR ePrint 2026/140.
- [9] National Institute of Standards and Technology, “FIPS 204: Module-Lattice-Based Digital Signature Standard,” 2024.
- [10] National Institute of Standards and Technology, “FIPS 205: Stateless Hash-Based Digital Signature Standard,” 2024.
- [11] J. Bos *et al.*, “CRYSTALS-Kyber: A CCA-Secure Module-Lattice-Based KEM,” *IEEE EuroS&P*, 2018.
- [12] G. Alagic *et al.*, “Status Report on the Fourth Round of the NIST Post-Quantum Cryptography Standardization Process,” NIST IR 8545, 2025.
- [13] E. Fujisaki and T. Okamoto, “Secure Integration of Asymmetric and Symmetric Encryption Schemes,” *CRYPTO*, 1999.
- [14] D. Hofheinz, K. Hövelmanns, and E. Kiltz, “A Modular Analysis of the Fujisaki–Okamoto Transformation,” *TCC*, 2017.
- [15] A. Langley, M. Hamburg, and S. Turner, “Elliptic Curves for Security,” RFC 7748, IETF, 2016.
- [16] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” RFC 8446, IETF, 2018.
- [17] D. Stebila, S. Fluhrer, and S. Gueron, “Hybrid Key Exchange in TLS 1.3,” draft-ietf-tls-hybrid-design, IETF, 2023.
- [18] K. Kwiatkowski *et al.*, “Post-Quantum Key Exchange X25519MLKEM768 for TLS 1.3,” draft-kwiatkowski-tls-ecdhe-mlkem, IETF, 2024.
- [19] K. Bhargavan, C. Jacomme, F. Kiefer, and R. Schmidt, “Formal Verification of the PQXDH Post-Quantum Key Agreement Protocol,” *IEEE S&P*, 2024.
- [20] Apple Security Engineering, “iMessage with PQ3: The New State of the Art in Quantum-Secure Messaging,” Apple Security Research, 2024.
- [21] K. Hövelmanns *et al.*, “Binding Security of Implicitly-Rejecting KEMs and Application to BIKE and HQC,” IACR ePrint 2024/1233, 2024.
- [22] J.-K. Zinzindohoué, K. Bhargavan, J. Protzenko, and B. Beurdouche, “HACL*: A Verified Modern Cryptographic Library,” *ACM CCS*, 2017.
- [23] Cryspen, “Verified ML-KEM (Kyber) in libcrux,” 2024.
- [24] J. B. Almeida *et al.*, “Formally Verifying Kyber: Episode V,” *CRYPTO*, 2024.
- [25] G. Barthe, B. Grégoire, S. Heraud, and S. Z. Béguelin, “Computer-Aided Security Proofs for the Working Cryptographer,” *CRYPTO*, 2011.
- [26] S. Meier, B. Schmidt, C. Cremers, and D. Basin, “The TAMARIN Prover for the Symbolic Analysis of Security Protocols,” *CAV*, 2013.
- [27] B. Blanchet, “Modeling and Verifying Security Protocols with the Applied Pi Calculus and ProVerif,” *Found. Trends Priv. Secur.*, 2016.
- [28] O. Reparaz, J. Balasch, and I. Verbauwhede, “Dude, is My Code Constant Time?” *DATE*, 2017.
- [29] N. Nethercote and J. Seward, “Valgrind: A Framework for Heavyweight Dynamic Binary Instrumentation,” *ACM PLDI*, 2007.
- [30] A. Langley, “ctgrind: Checking that Functions are Constant Time with Valgrind,” 2010.
- [31] National Institute of Standards and Technology, “Automated Cryptographic Validation Protocol (ACVP),” <https://github.com/usnistgov/ACVP>, 2024.
- [32] J. Birr-Pixton *et al.*, “rustls: A Modern TLS Library in Rust,” <https://github.com/rustls/rustls>, 2024.

ACKNOWLEDGMENT

The author thanks the maintainers of libcrux/HACL*, EasyCrypt, Tamarin, and ProVerif, whose tools this work builds upon. *[Funding/advisor acknowledgments to be added.]*

APPENDIX A ARTIFACT AVAILABILITY

The complete suite—Rust core and backends, the five language faces, the EasyCrypt/Tamarin/ProVerif developments, the constant-time and netem harnesses, and the figure-generation scripts—is open source at <https://github.com/billlza/q-periapt>. All numbers in this paper are reproducible from that repository; the bare-metal netem protocol is provided as a script.